

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)
```

```
df = pd.read_csv("diabetes.csv")
```

```
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	Pedigree	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies           768 non-null   int64
1   Glucose                768 non-null   int64
2   BloodPressure         768 non-null   int64
3   SkinThickness         768 non-null   int64
4   Insulin                768 non-null   int64
```

```

5    BMI          768 non-null    float64
6    Pedigree     768 non-null    float64
7    Age          768 non-null    int64
8    Outcome      768 non-null    int64

```

```
dtypes: float64(2), int64(7)
```

```
memory usage: 54.1 KB
```

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness
Insulin \				
count	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458
std	3.369578	31.972618	19.355807	15.952218
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000
75%	6.000000	140.250000	80.000000	32.000000
max	17.000000	199.000000	122.000000	99.000000

	BMI	Pedigree	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

```
cols = ["Glucose", "BloodPressure", "SkinThickness",
        "Insulin", "BMI"]
```

```
df[cols] = df[cols].replace(0, np.nan)
```

```
for col in cols:
    df[col] = df[col].fillna(df[col].median())
```

```
X = df.drop("Outcome", axis=1)
```

```
y = df["Outcome"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```

    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

KNeighborsClassifier(n_neighbors=5)
KNeighborsClassifier()

knn = KNeighborsClassifier()

param_grid = {
    "n_neighbors": range(1, 31),
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan"]
}

grid = GridSearchCV(
    knn,
    param_grid,
    cv=5,
    scoring="recall",
    n_jobs=-1
)

grid.fit(X_train_scaled, y_train)

GridSearchCV(cv=5, estimator=KNeighborsClassifier(), n_jobs=-1,
             param_grid={'metric': ['euclidean', 'manhattan'],
                          'n_neighbors': range(1, 31),
                          'weights': ['uniform', 'distance']}},
             scoring='recall')

print(grid.best_params_)
{'metric': 'euclidean', 'n_neighbors': 17, 'weights': 'distance'}

print(grid.best_score_)
0.6167220376522702

best_knn = grid.best_estimator_

y_pred = best_knn.predict(X_test_scaled)

```

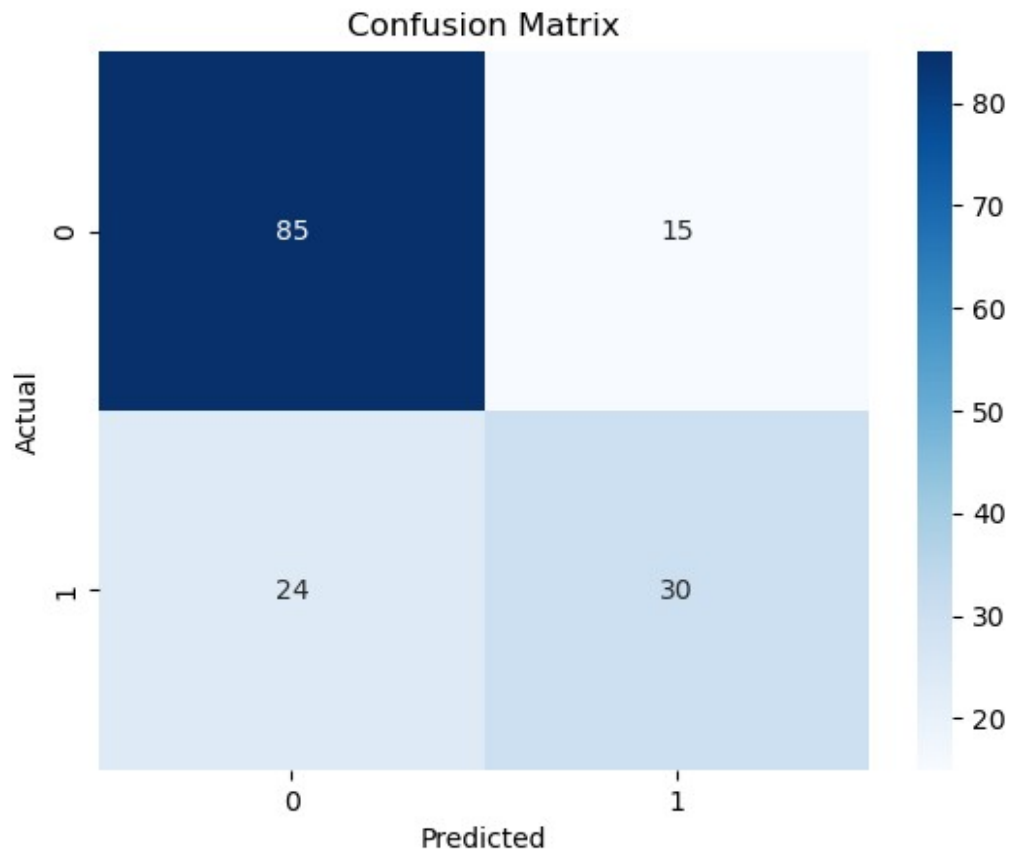
```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
Accuracy: 0.7467532467532467
precision = precision_score(y_test, y_pred)
print("Precision:", precision)
Precision: 0.6666666666666666
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
Recall: 0.5555555555555556
f1 = f1_score(y_test, y_pred)
print("F1 Score:", f1)
F1 Score: 0.6060606060606061
error_rate = 1 - accuracy
print("Error Rate:", error_rate)
Error Rate: 0.2532467532467533
cm = confusion_matrix(y_test, y_pred)
print(cm)
[[85 15]
 [24 30]]

import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```



```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.78	0.85	0.81	100
1	0.67	0.56	0.61	54
accuracy			0.75	154
macro avg	0.72	0.70	0.71	154
weighted avg	0.74	0.75	0.74	154